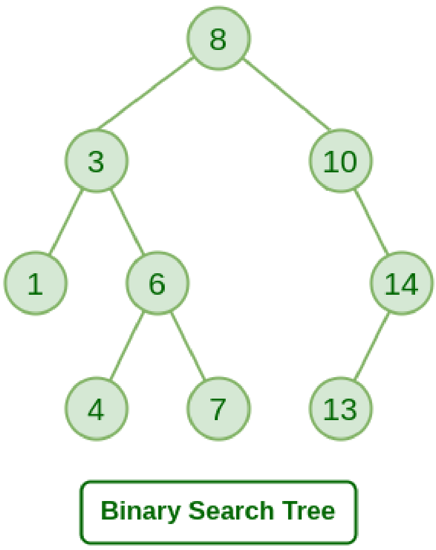
# Applications, Advantages and Disadvantages of Binary Search Tree

Last Updated : 30 Jul, 2024

A Binary Search Tree **(BST)** is a data structure used to storing data in a sorted manner. Each node in a **Binary Search Tree** has at most two children, a **left child** and a **right child**, with the **left child** containing values less than the **parent node** and the **right child** containing values greater than the **parent node**. This hierarchical structure allows for efficient **searching, insertion, and deletion** operations on the data stored in the tree.




*Binary Search Tree*

## Applications of Binary Search Tree (BST):

Please refer Applications of BSTs for detailed explanation.

## Advantages of Binary Search Tree (BST):

- **Efficient searching:** O(log n) time complexity for searching with a self balancing BST
- **Ordered structure:** Elements are stored in sorted order, making it easy to find the next or previous element
- **Dynamic insertion and deletion:** Elements can be added or removed efficiently
- **Balanced structure:** Balanced BSTs maintain a logarithmic height, ensuring efficient operations

- **Doubly Ended Priority Queue**: In BSTs, we can maintain both maximum and minimum efficiently

## Disadvantages of Binary Search Tree (BST):

- **Not self-balancing:** Unbalanced BSTs can lead to poor performance
- **Worst-case time complexity:** In the worst case, BSTs can have a linear time complexity for searching and insertion
- **Memory overhead:** BSTs require additional memory to store pointers to child nodes
- **Not suitable for large datasets:** BSTs can become inefficient for very large datasets
- **Limited functionality:** BSTs only support searching, insertion, and deletion operations

The main competitor Data Structure of BST is Hash Table in terms of applications. We have discussed BST vs Hash Table in details for your reference.



Next Article

Insertion in Binary Search Tree (BST)

Binary Search Tree (BST) is a data structure that is commonly used to implement efficient searching, insertion, and deletion operations along with maintaining sorted sequence of data. Please remember th...

3 min read

## Applications, Advantages and Disadvantages of Binary Search Tree

A Binary Search Tree (BST) is a data structure used to storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right child, with the left child containing...

2 min read

## Insertion in Binary Search Tree (BST)

Given a BST, the task is to insert a new node in this BST.Example: How to Insert a value in a Binary Search Tree:A new key is always inserted at the leaf by maintaining the property of the binary search...

15 min read

## Searching in Binary Search Tree (BST)

Given a BST, the task is to search a node in this BST. For searching a value in BST, consider it as a sorted array. Now we can easily perform search operation in BST using Binary Search Algorithm. Input: Root of...

7 min read

## Deletion in Binary Search Tree (BST)

Given a BST, the task is to delete a node in this BST, which can be broken down into 3 scenarios:Case 1. Delete a Leaf Node in BSTDeletion in BSTCase 2. Delete a Node with Single Child in BSTDeleting a...

10 min read

## Binary Search Tree (BST) Traversals â€“ Inorder, Preorder, Post Order

Given a Binary Search Tree, The task is to print the elements in inorder, preorder, and postorder traversal of the Binary Search Tree.Â Input:Â A Binary Search TreeOutput:Â Inorder Traversal: 10 20 30 100 150...

10 min read

## Balance a Binary Search Tree

Given a BST (Binary Search Tree) that may be unbalanced, the task is to convert it into a balanced BST that has the minimum possible height.Examples: Input: Output: Explanation: The above unbalanced BST...

10 min read

## Self-Balancing Binary Search Trees

Self-Balancing Binary Search Trees are height-balanced binary search trees that automatically keep the height as small as possible when insertion and deletion operations are performed on the tree. The heigh...

4 min read